

АНАЛИЗ НА НАДЕЖДНОСТТА

на софтуерна система

Система	Android приложение
Платформа	Android (minSdk 24, targetSdk 35)
Език / Framework	Kotlin 2.2 / Jetpack Compose
Метод на анализ	SRGM чрез MLE (Levenberg-Marquardt)
Анализирани модели	5 SRGM модела
Критерии за оценка	R^2 , AIC, BIC

1. Въведение

Под надеждност на софтуера разбираме вероятността дадена програма да работи без откази в рамките на определен интервал от време и при конкретни условия на експлоатация [1]. В практиката тя е едно от централните мерки за качество – особено там, където последствията от повреда са сериозни.

В този документ е представен анализът на надеждността на Android приложението за статистическа обработка на тестови данни, написано в Kotlin с Jetpack Compose. Приложението реализира пет SRGM модела и позволява зареждане на CSV данни, оценка на параметри чрез MLE и преглед на резултатите чрез графики.

2. Описание на анализираната система

Приложението следва MVVM архитектура и е разделен на три логически слоя:

- **domain** – дефиниции на данни (ReliabilityTestRecord, ModelParameters) и абстракция SRGMEngine.
- **logic** – реализации на петте SRGM модела (SRGMEngines.kt), ReportGenerator.kt и PdfExporter.kt.
- **ui** – Compose компоненти (BentoBox, ReliabilityGraph, Scorecard), BBCViewModel и Theme.kt.

От функционална гледна точка приложението позволява: зареждане на времеви редове от откази, филтриране по времеви диапазон, паралелно пресмятане на параметри за петте модела чрез алгоритъма Levenberg-Marquardt, изчисляване на R^2 , AIC и BIC, преглед на кривите в графика и генериране на PDF или текстов доклад.

3. Теоретична основа на SRGM

SRGM моделите описват как нараства надеждността на даден софтуер с напредването на тестването – чрез стохастична връзка между броя засечени грешки и отминалото тестово време [2, 3]. Голяма част от тях се основават на NHPP (Non-Homogeneous Poisson Process), при който средният кумулативен брой грешки $\mu(t)$ се задава с аналитична формула.

3.1 Имплементирани модели

Модел	Формула $\mu(t)$	Параметри	Тип крива
Goel-Okumoto (GO)	$a(1 - e^{(-bt)})$	a: общ брой грешки b: скорост на откр.	Експоненциална
Jelinski-Moranda (JM)	$a(1 - e^{(-bt)})$	a: начален брой грешки b: скорост на отстр.	Експоненциална
Inflection S-Shaped	$a(1 - e^{(-bt)}) / (1 + \phi e^{(-bt)})$	a, b: базови φ: инфлекссионен фактор	S-образна с инфлексия
Yamada Exponential	$a[1 - (1 + bt)e^{(-bt)}]$	a: брой грешки b: скорост на тестване	Ранно S-образна
Delayed S-Shaped	$a[1 - (1 + bt)e^{(-bt)}]$	a: брой грешки b: забавяне при откр.	Забавена S-образна

Таблица 1. Математически формули на реализираните SRGM модели.

3.2 Goel-Okumoto (GO) модел

Goel и Okumoto го предлагат през 1979 г. [4] и той се приема за базов NHPP модел. Параметърът а дава очаквания общ брой грешки в системата, а b – скоростта, с която те се намират по време на тестване:

$$\mu(t) = a \cdot (1 - e^{(-bt)})$$

3.3 Jelinski-Moranda (JM) модел

Разработен от Jelinski и Moranda през 1972 г. [5], JM е сред по-ранните SRGM и се базира на марковски процес. Изхожда от допускането, че всяка грешка има еднаква вероятност да се прояви. Математически формата е идентична с GO, но различните начални условия в MLE оценката дават различни стойности на параметрите.

3.4 Inflection S-Shaped модел

Ohba го представя през 1984 г. [6] с цел да улови кривата на научаване при тестване. Параметърът ϕ (phi) определя местоположението на инфлексивната точка. Когато $\phi \rightarrow 0$, моделът се свежда до GO:

$$\mu(t) = a \cdot (1 - e^{(-bt)}) / (1 + \phi \cdot e^{(-bt)})$$

3.5 Yamada Exponential и Delayed S-Shaped

И двата модела са разработени от Yamada et al. (1983) [7] на основата на S-образна функция. При Yamada Exponential началните условия са $a_0 = 1.2 \cdot N$, $b_0 = 0.05$, докато Delayed S-Shaped е настроен по-консервативно ($a_0 = 1.5 \cdot N$, $b_0 = 0.03$):

$$\mu(t) = a \cdot [1 - (1 + bt) \cdot e^{(-bt)}]$$

4. Метод за оценка на параметри – MLE с Levenberg-Marquardt

За оценка на параметрите и в петте модела е използван методът на максималното правдоподобие (MLE). Той максимизира логаритмичната функция:

$$\ln L(\theta) = -(n/2) \cdot \ln(2\pi\sigma^2) - SSE/(2\sigma^2)$$

На практика проблемът се преформулира като нелинейни най-малки квадрати, при което се минимизира:

$$SSE = \sum_i [\mu(t_i; \theta) - N_i]^2$$

Алгоритъмът Levenberg-Marquardt (LMA), реализиран чрез Apache Commons Math 3 (LevenbergMarquardtOptimizer) [8], съчетава подходите на Гаус-Нютон и стийпенс-дисцент. В приложението е конфигуриран по следния начин:

- $\text{maxEvaluations} = 5\,000$ (максимален брой изчисления на целевата функция)
- $\text{maxIterations} = 5\,000$ (максимален брой итерации)
- Якобиановата матрица J се изчислява аналитично за всеки модел – точно от числено диференциране.
- Начални условия: $\theta_0 = [1.2 \cdot N_{\text{max}}, 0.05]$ (GO/JM/Yamada) или $[1.5 \cdot N_{\text{max}}, 0.03]$ (Delayed).

5. Статистически критерии за качество на прилягането

Проекта изчислява три статистически критерия за сравнение на моделите [2, 9]:

Метрика	Определение	Интерпретация
R^2 (Коеф. на детерминация)	$1 - \text{SSE}/\text{SST}$	$\geq 0.9 \rightarrow$ отлично $0.7-0.9 \rightarrow$ добро $< 0.7 \rightarrow$ слабо
AIC (Akaike)	$2k - 2\ln(L)$	По-малко = по-добре Наказва сложността
BIC (Bayesian)	$k \cdot \ln(n) - 2\ln(L)$	По-строг от AIC при големи извадки
SSE	$\sum (y_i - \hat{y}_i)^2$	Сума на кв. на остатъците

Таблица 2. Статистически метрики за качество на прилягането.

AIC и BIC са информационни критерии, при които се балансира между качеството на прилягане ($-2\ln L$) и сложността на модела (k параметъра). BIC наказва по-строго при по-голям брой наблюдения n , защото $k \cdot \ln(n)$ нараства по-бързо от $2k$ в AIC [9]. Стойност $R^2 \geq 0.90$ е индикатор за отлично прилягане; когато два модела са близки ($\Delta\text{AIC} < 2$), логично е да се избере по-опростеният.

6. Сравнителен анализ на моделите

Модел	Тип	k	Предимства	Ограничения
Goel-Okumoto	NHPP	2	Прост; добър за експ. данни	Не улавя S-образни тенденции
Jelinski-Moranda	Марков	2	Класически; широко използван	Предполага еднакви грешки
Inflection S-Shaped	NHPP	3	Улавя кривата на научаване	Рискува пренастройване
Yamada Exponential	NHPP	2	Реалистичен; S-образен без 3-ти парам.	По-бавна конвергенция
Delayed S-Shaped	NHPP	2	Моделира закъснение при тестване	Може да се слее с Yamada

Таблица 3. Сравнение на имплементираните SRGM модела.

Процедурата за избор на модел е: (1) изчисляват се AIC и BIC за всичките пет модела; (2) за финалист се взима моделът с най-ниска AIC при условие $R^2 \geq 0.90$; (3) ако разликата между два модела е $\Delta AIC < 2$, по-простият (2-параметричен) е за предпочитане [9].

7. Обработка и валидиране на входните данни

За зареждане на данни са предвидени два начина:

- **CSV файл** – парсиран с Apache Commons CSV 1.11. Очаква колони 'Time' и 'Failures'. Невалидните редове се пропускам (mapNotNull).
- **URL адрес** – HTTP GET заявка в IO dispatcher. Поддържа формата на NASA/DACS (колони 'CPU_hour', 'Failures').

Ако резултатът е празен, AppState минава в ERROR. Потребителят може да стеснява времевия диапазон на данните чрез слайдер; при всяка промяна анализът се преизчислява с 200 ms debounce, за да не се натоварва излишно UI нишката.

8. Изводи от анализа

8.1 Обхват на моделирането

Реализираните модели покриват пет SRGM класа: два NHPP с монотонно намаляваща интензивност (GO, JM), един NHPP с инфлексивен параметър (IS-S) и два NHPP със S-образна функция (Yamada, Delayed). Тази комбинация дава достатъчна гъвкавост при работа с различни типове реални данни от тестване.

8.2 Качество на оценката на параметрите

Решението да се използва аналитичен якобиан вместо числено диференциране се отразява видимо на конвергенцията – итерациите са по-малко и числените грешки са значително по-ниски. Лимитът от 5 000 итерации на практика не се достига при стандартни SRGM набори от данни (20–200 точки).

8.3 Изборни критерии за модел

Едновременно разглеждане на R^2 , AIC и BIC намалява риска от субективен или случаен избор на модел. При малки извадки ($n < 30$) е разумно да се разчита повече на BIC, тъй като той налага по-строго наказание за допълнителна сложност.

8.4 Ограничения

Текущите доверителни интервали са изчислени като прости $\pm 5\%$ отклонения – не са базирани на Хесиана и не представляват строги статистически граници. Освен това, всички модели приемат независимост на грешките помежду им, което при систематични дефекти може да е нереалистично допускане.

8.5 Препоръки

Като следваща стъпка би трябвало да се въведе разделяне на данните на training и test подмножества, след което да се измери предиктивната точност на избрания модел. Ако са налични данни от няколко версии на системата, йерархичен байесов подход може да даде по-стабилни и по-добре обосновани оценки.

9. Използвани литературни източници и интернет ресурси

- [1] IEEE Std 1633-2016. "IEEE Recommended Practice on Software Reliability." IEEE Computer Society, 2017. DOI: 10.1109/IEEESTD.2017.7827907
- [2] Lyu, M.R. (Ed.). "Handbook of Software Reliability Engineering." McGraw-Hill / IEEE Computer Society Press, 1996. ISBN: 0-07-039400-8. Online: <https://www.cse.cuhk.edu.hk/~lyu/book/reliability/>
- [3] Musa, J.D., Iannino, A., Okumoto, K. "Software Reliability: Measurement, Prediction, Application." McGraw-Hill, 1987. ISBN: 0-07-044093-X
- [4] Goel, A.L. and Okumoto, K. (1979). "Time-Dependent Error-Detection Rate Model for Software Reliability." IEEE Transactions on Reliability, 28(3), pp. 206–211. DOI: 10.1109/TR.1979.5220566
- [5] Jelinski, Z. and Moranda, P.B. (1972). "Software Reliability Research." In: W. Freiberger (Ed.), Statistical Computer Performance Evaluation, Academic Press, pp. 465–484.
- [6] Ohba, M. (1984). "Inflection S-shaped Software Reliability Growth Model." In: Stochastic Models in Reliability Theory, Springer-Verlag, pp. 144–162. DOI: 10.1007/978-3-642-45587-2_10
- [7] Yamada, S., Ohba, M., Osaki, S. (1983). "S-shaped Reliability Growth Modeling for Software Error Detection." IEEE Transactions on Reliability, 32(5), pp. 475–484. DOI: 10.1109/TR.1983.5221735
- [8] Apache Software Foundation. "Apache Commons Math 3.6.1 – LevenbergMarquardtOptimizer." <https://commons.apache.org/proper/commons-math/javadocs/api-3.6.1/> [Достъп: юни 2026]
- [9] Burnham, K.P. and Anderson, D.R. "Model Selection and Multimodel Inference." 2nd ed., Springer-Verlag, 2002. ISBN: 978-0-387-95364-9
- [10] NASA / DACS. "Software Reliability – Historical Failure Data." <https://www.thedacs.com/databases/sre/> [Достъп: юни 2026]
- [11] Pham, H. "Software Reliability." Springer-Verlag, 2000. ISBN: 978-981-4021-75-6
- [12] Android Developers. "Android Architecture Components – ViewModel." <https://developer.android.com/topic/libraries/architecture/viewmodel> [Достъп: юни 2026]
- [13] Kotlin Documentation. "Coroutines guide." <https://kotlinlang.org/docs/coroutines-guide.html> [Достъп: юни 2026]